Lab 3: Clustering (Customer Segmentation)

Duration: 100 minutes **Course:** Data Mining **Lab Type:** Guided practical session

Learning Objectives

By the end of this lab, you should be able to:

- Understand the structure and quality of a customer dataset
- Perform comprehensive preprocessing for clustering analysis
- Visualize data before and after clustering using dimensionality reduction
- Apply and compare multiple clustering algorithms (K-Means, DBSCAN, GMM)
- Evaluate clustering results using both internal and external validation metrics
- Interpret and discuss the practical implications of clustering results

Experimental Background

Clustering is an **unsupervised learning** task that aims to group similar data points together without using class labels during training. In this lab, you will perform customer segmentation analysis and compare the clustering results with predefined customer segment labels (if available).

Part 1: Introduction and Research Questions

Key Questions to Consider

1. **Why is clustering useful in customer analysis?**
    - Customer segmentation for targeted marketing
    - Identifying customer behavior patterns
    - Optimizing resource allocation based on customer groups
2. **What is the fundamental difference between classification and clustering?**
    - Classification: Supervised learning with known labels
    - Clustering: Unsupervised learning without predefined labels
3. **Why is preprocessing crucial before clustering?**
    - Handling mixed data types (categorical vs numerical)
    - Standardization for distance-based algorithms
    - Dimensionality reduction for visualization and performance

Part 2: Data Understanding and Preprocessing

2.1 Data Loading and Initial Exploration

In this section, we load the customer segmentation dataset and perform initial data exploration to understand its structure and quality.

In [2]:
```
# Import necessary libraries
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (
    silhouette_score,
    adjusted_rand_score,
    normalized_mutual_info_score,
    davies_bouldin_score
)

# Set display options for better readability
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
plt.style.use('seaborn-v0_8')

print("✅ Libraries imported successfully")
```

```
✅ Libraries imported successfully
```

2.2 Data Quality Assessment

In this section, we assess the quality of the dataset by:

- Checking for missing values and duplicates
- Examining data distributions and summary statistics
- Identifying potential outliers and data inconsistencies

In [3]:
```
# Load and explore the dataset
print("📊 Loading customer segmentation dataset...")

# Load the dataset
file_path = "../lab_03/customer_segmentation_cleaned.csv"
df = pd.read_csv(file_path)

print(f"✅ Dataset loaded successfully")
print(f"- Dataset shape: {df.shape}")
print(f"- Number of customers: {len(df)}")
print(f"- Number of features: {len(df.columns)}")

print("\n📋 Dataset Overview:")
print("First 5 rows:")
display(df.head())
```

```python
print("\n🔍 Column Information:")
print(df.info())

print("\n🎯 Target Variable Analysis:")
print(f"- Target column: Segmentation")
print(f"- Unique segments: {df['Segmentation'].unique()}")
print(f"- Segment distribution:")
print(df['Segmentation'].value_counts())
```

```
📊 Loading customer segmentation dataset...
✅ Dataset loaded successfully
- Dataset shape: (10673, 12)
- Number of customers: 10673
- Number of features: 12

📋 Dataset Overview:
First 5 rows:
```

	Gender	Ever_Married	Age	Graduated	Profession	Work_Experience	Spendin
0	Male	No	22	No	Healthcare	1.0	
1	Female	Yes	38	Yes	Engineer	1.0	
2	Female	Yes	67	Yes	Engineer	1.0	
3	Male	Yes	67	Yes	Lawyer	0.0	
4	Female	Yes	40	Yes	Entertainment	1.0	

```
🔍 Column Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10673 entries, 0 to 10672
Data columns (total 12 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Gender                 10673 non-null  object
 1   Ever_Married           10673 non-null  object
 2   Age                    10673 non-null  int64
 3   Graduated              10673 non-null  object
 4   Profession             10673 non-null  object
 5   Work_Experience        10673 non-null  float64
 6   Spending_Score         10673 non-null  object
 7   Family_Size            10673 non-null  float64
 8   Var_1                  10673 non-null  object
 9   Segmentation           10673 non-null  object
 10  AgeGroup               10673 non-null  object
 11  Marital_Status_Normal  10673 non-null  object
dtypes: float64(2), int64(1), object(9)
memory usage: 1000.7+ KB
None

🎯 Target Variable Analysis:
- Target column: Segmentation
- Unique segments: ['D' 'A' 'B' 'C']
- Segment distribution:
Segmentation
D    3016
A    2811
C    2438
B    2408
Name: count, dtype: int64
```

In [4]:

```python
# Comprehensive data quality assessment
print("🔍 Performing Data Quality Assessment...")

print("\n📊 Data Types:")
print(df.dtypes)

print("\n❌ Missing Values Analysis:")
missing_data = df.isnull().sum()
print("Missing values per column:")
for col, missing_count in missing_data.items():
    if missing_count > 0:
        print(f"  - {col}: {missing_count} missing values ({missing_count/len(df
    else:
        print(f"  - {col}: No missing values ✓")

print("\n🔁 Duplicate Records:")
duplicate_count = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_count} ({duplicate_count/len(df)*10

print("\n📈 Summary Statistics:")
print("Numerical features:")
display(df.describe())

print("\n📊 Categorical Features Summary:")
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    print(f"\n{col}:")
```

```
    print(f"  - Unique values: {df[col].nunique()}")
    print(f"  - Most frequent: {df[col].mode()[0]} (count: {df[col].value_counts

print("\n✅ DATA QUALITY SUMMARY:")
print(f"- Total records: {len(df)}")
print(f"- Missing values: {missing_data.sum()} (all columns complete ✓)")
print(f"- Duplicate rows: {duplicate_count} (to be handled in preprocessing)")
print(f"- Categorical features: {len(categorical_cols)}")
print(f"- Numerical features: {len(df.select_dtypes(include=['number']).columns)
```

```
🔍 Performing Data Quality Assessment...

📊 Data Types:
Gender                    object
Ever_Married              object
Age                        int64
Graduated                 object
Profession                object
Work_Experience          float64
Spending_Score            object
Family_Size              float64
Var_1                     object
Segmentation              object
AgeGroup                  object
Marital_Status_Normal     object
dtype: object

❌ Missing Values Analysis:
Missing values per column:
  - Gender: No missing values ✓
  - Ever_Married: No missing values ✓
  - Age: No missing values ✓
  - Graduated: No missing values ✓
  - Profession: No missing values ✓
  - Work_Experience: No missing values ✓
  - Spending_Score: No missing values ✓
  - Family_Size: No missing values ✓
  - Var_1: No missing values ✓
  - Segmentation: No missing values ✓
  - AgeGroup: No missing values ✓
  - Marital_Status_Normal: No missing values ✓

🔁 Duplicate Records:
Number of duplicate rows: 787 (7.4%)

📈 Summary Statistics:
Numerical features:
```

	Age	Work_Experience	Family_Size
count	10673.000000	10673.000000	10673.000000
mean	43.517287	2.456385	2.851026
std	16.765221	3.251340	1.505024
min	18.000000	0.000000	1.000000
25%	30.000000	0.000000	2.000000
50%	41.000000	1.000000	3.000000
75%	53.000000	4.000000	4.000000
max	89.000000	14.000000	9.000000

```
📊 Categorical Features Summary:

Gender:
  - Unique values: 2
  - Most frequent: Male (count: 5824)

Ever_Married:
  - Unique values: 2
  - Most frequent: Yes (count: 6151)

Graduated:
  - Unique values: 2
  - Most frequent: Yes (count: 6661)

Profession:
  - Unique values: 9
  - Most frequent: Artist (count: 3472)

Spending_Score:
  - Unique values: 3
  - Most frequent: Low (count: 6479)

Var_1:
  - Unique values: 7
  - Most frequent: Cat_6 (count: 6998)

Segmentation:
  - Unique values: 4
  - Most frequent: D (count: 3016)

AgeGroup:
  - Unique values: 6
  - Most frequent: 26-35 (count: 2556)

Marital_Status_Normal:
  - Unique values: 2
  - Most frequent: Normal (count: 9897)

✅ DATA QUALITY SUMMARY:
- Total records: 10673
- Missing values: 0 (all columns complete ✓)
- Duplicate rows: 787 (to be handled in preprocessing)
- Categorical features: 9
- Numerical features: 3
```

2.3 Data Preprocessing Pipeline

This section covers the complete preprocessing workflow:

- **Feature Selection**: Identifying relevant features for clustering
- **Data Cleaning**: Handling duplicates and missing values
- **Categorical Encoding**: Converting categorical variables to numerical format
- **Feature Scaling**: Standardizing numerical features for clustering algorithms
- **Dimensionality Reduction**: Using PCA for visualization and analysis

In [5]:
```
# Prepare data for clustering analysis
print("🔧 Preparing Data for Clustering...")

# Identify target variable for external validation
label_column = "Segmentation"
print(f"🎯 Target variable identified: {label_column}")
print(f"- Unique segments: {df[label_column].unique()}")
print(f"- Segment distribution:")
segment_counts = df[label_column].value_counts()
for segment, count in segment_counts.items():
    print(f"  - {segment}: {count} customers ({count/len(df)*100:.1f}%)")

# Separate features and target
y_true = df[label_column].copy()
X_df = df.drop(columns=[label_column]).copy()

print(f"\n📊 Feature-Target Separation:")
print(f"- Features shape: {X_df.shape}")
print(f"- Target variable shape: {y_true.shape}")
print(f"- Feature columns: {list(X_df.columns)}")

print("\n✅ Data preparation completed successfully")
```

```
🔧 Preparing Data for Clustering...
🎯 Target variable identified: Segmentation
- Unique segments: ['D' 'A' 'B' 'C']
- Segment distribution:
  - D: 3016 customers (28.3%)
  - A: 2811 customers (26.3%)
  - C: 2438 customers (22.8%)
  - B: 2408 customers (22.6%)

📊 Feature-Target Separation:
- Features shape: (10673, 11)
- Target variable shape: (10673,)
- Feature columns: ['Gender', 'Ever_Married', 'Age', 'Graduated', 'Profession',
'Work_Experience', 'Spending_Score', 'Family_Size', 'Var_1', 'AgeGroup', 'Marital
_Status_Normal']

✅ Data preparation completed successfully
```

2.4 Data Preprocessing Implementation

In this section, we implement the complete preprocessing pipeline including:

- Handling categorical variables through encoding

- Standardizing numerical features
- Applying dimensionality reduction for visualization

In [6]:
```
# Handle duplicate records before preprocessing
print("🔧 Handling Duplicate Records...")
original_count = len(X_df)
X_df = X_df.drop_duplicates()
y_true = y_true[X_df.index]  # Keep corresponding labels

print(f"- Original records: {original_count}")
print(f"- After removing duplicates: {len(X_df)}")
print(f"- Duplicates removed: {original_count - len(X_df)} ({((original_count -

print("\n✅ Duplicate handling completed")
```

```
🔧 Handling Duplicate Records...
- Original records: 10673
- After removing duplicates: 9062
- Duplicates removed: 1611 (15.1%)

✅ Duplicate handling completed
```

In [7]:
```
# Complete preprocessing pipeline
print("🔧 Implementing Complete Preprocessing Pipeline...")

# Create a copy for preprocessing
X_processed = X_df.copy()

print("\n📊 Categorical Feature Encoding:")
encoders = {}
categorical_cols = X_processed.select_dtypes(include=['object']).columns

for col in categorical_cols:
    le = LabelEncoder()
    X_processed[col] = le.fit_transform(X_processed[col].astype(str))
    encoders[col] = le
    print(f"  - {col}: {len(le.classes_)} unique values encoded")

print(f"\n✅ Categorical encoding completed - {len(encoders)} features encoded"

print("\n📊 Feature Standardization:")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_processed)
X_scaled_df = pd.DataFrame(X_scaled, columns=X_processed.columns)

print("✅ Feature standardization completed")

print("\n📊 Dimensionality Reduction (PCA) for Visualization:")
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(f"- Explained variance ratio: {pca.explained_variance_ratio_}")
print(f"- Total variance explained: {pca.explained_variance_ratio_.sum():.1%}")

print("\n📋 Processed Data Overview:")
print(f"- Final data shape: {X_scaled.shape}")
print(f"- Number of features: {X_scaled.shape[1]}")
print(f"- Number of samples: {X_scaled.shape[0]}")
```

```
print("\n✅ PREPROCESSING PIPELINE COMPLETED SUCCESSFULLY")
```

```
🔧 Implementing Complete Preprocessing Pipeline...

📊 Categorical Feature Encoding:
  - Gender: 2 unique values encoded
  - Ever_Married: 2 unique values encoded
  - Graduated: 2 unique values encoded
  - Profession: 9 unique values encoded
  - Spending_Score: 3 unique values encoded
  - Var_1: 7 unique values encoded
  - AgeGroup: 6 unique values encoded
  - Marital_Status_Normal: 2 unique values encoded

✅ Categorical encoding completed - 8 features encoded

📊 Feature Standardization:
✅ Feature standardization completed

📊 Dimensionality Reduction (PCA) for Visualization:
- Explained variance ratio: [0.25623061 0.14074737]
- Total variance explained: 39.7%

📋 Processed Data Overview:
- Final data shape: (9062, 11)
- Number of features: 11
- Number of samples: 9062

✅ PREPROCESSING PIPELINE COMPLETED SUCCESSFULLY
```

Part 3: Clustering Algorithms Implementation

3.1 K-Means Clustering

K-Means is a centroid-based clustering algorithm that partitions data into K clusters by minimizing the within-cluster variance. It works well when clusters are spherical and well-separated.

3.1.1 Determining Optimal Number of Clusters (Elbow Method)

The elbow method helps determine the optimal number of clusters by finding the point where the inertia (within-cluster sum of squares) starts decreasing at a slower rate.

In [8]:
```
# Determine optimal K using elbow method and silhouette analysis
print("🔍 Determining Optimal Number of Clusters for K-Means...")

k_range = range(2, 11)
inertia = []
silhouette_scores = []

print("\n📊 Evaluating different K values:")
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(X_scaled)
```

```
    inertia.append(kmeans.inertia_)
    silhouette_avg = silhouette_score(X_scaled, cluster_labels)
    silhouette_scores.append(silhouette_avg)
    print(f"  - k={k}: Inertia = {kmeans.inertia_:.0f}, Silhouette = {silhouette

print("\n✅ K evaluation completed")
```

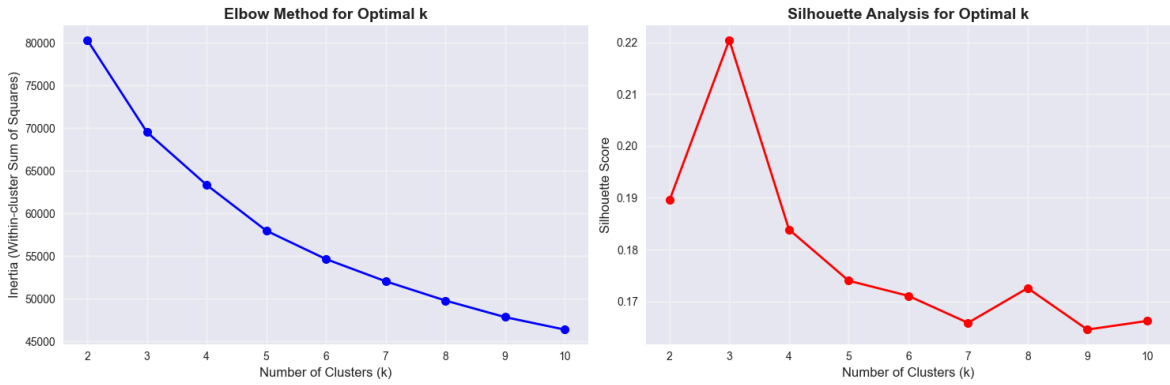
```
🔍 Determining Optimal Number of Clusters for K-Means...

📊 Evaluating different K values:
  - k=2: Inertia = 80315, Silhouette = 0.190
  - k=3: Inertia = 69523, Silhouette = 0.220
  - k=4: Inertia = 63337, Silhouette = 0.184
  - k=5: Inertia = 57977, Silhouette = 0.174
  - k=6: Inertia = 54650, Silhouette = 0.171
  - k=7: Inertia = 52050, Silhouette = 0.166
  - k=8: Inertia = 49798, Silhouette = 0.173
  - k=9: Inertia = 47857, Silhouette = 0.165
  - k=10: Inertia = 46393, Silhouette = 0.166

✅ K evaluation completed
```

3.1.2 Visualizing Elbow Method and Silhouette Analysis

Visualizing both inertia and silhouette scores helps in making an informed decision about the optimal number of clusters.

In [9]:
```
# Visualize elbow method and silhouette analysis
print("📊 Creating Visualization for Cluster Selection...")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Plot 1: Elbow method (inertia)
ax1.plot(k_range, inertia, 'bo-', linewidth=2, markersize=8)
ax1.set_xlabel('Number of Clusters (k)', fontsize=12)
ax1.set_ylabel('Inertia (Within-cluster Sum of Squares)', fontsize=12)
ax1.set_title('Elbow Method for Optimal k', fontsize=14, fontweight='bold')
ax1.grid(True, alpha=0.3)

# Plot 2: Silhouette scores
ax2.plot(k_range, silhouette_scores, 'ro-', linewidth=2, markersize=8)
ax2.set_xlabel('Number of Clusters (k)', fontsize=12)
ax2.set_ylabel('Silhouette Score', fontsize=12)
ax2.set_title('Silhouette Analysis for Optimal k', fontsize=14, fontweight='bold
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("✅ Visualization created successfully")
```

```
📊 Creating Visualization for Cluster Selection...
```




```
✅ Visualization created successfully
```

3.1.3 Selecting Optimal K and Implementing K-Means

Based on the analysis, we select the optimal number of clusters and implement the K-Means algorithm.

In [10]:

```
# Select optimal K and implement K-Means
print("🎯 Selecting Optimal Number of Clusters...")

# Find optimal k based on silhouette score
optimal_k = k_range[np.argmax(silhouette_scores)]
print(f"- Optimal k based on silhouette score: {optimal_k}")
print(f"- Maximum silhouette score: {max(silhouette_scores):.3f}")

# Also check elbow point (where the slope changes significantly)
elbow_k = None
for i in range(1, len(inertia)-1):
    if abs(inertia[i] - inertia[i-1]) / abs(inertia[i+1] - inertia[i]) > 2:
        elbow_k = k_range[i]
        break

if elbow_k:
    print(f"- Potential elbow point at k: {elbow_k}")
    print(f"- Final decision: Using k={optimal_k} (silhouette-optimized)")

print("\n🔧 Implementing K-Means Clustering...")

# Fit K-Means with optimal k
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X_scaled)

print(f"✅ K-Means clustering completed with k={optimal_k}")

print("\n📊 Cluster Distribution:")
unique, counts = np.unique(y_kmeans, return_counts=True)
for cluster, count in zip(unique, counts):
    print(f"  - Cluster {cluster}: {count} customers ({count/len(y_kmeans)*100:.

print("\n📈 Performance Metrics:")
print(f"  - Silhouette Score: {silhouette_score(X_scaled, y_kmeans):.3f}")
print(f"  - Davies-Bouldin Index: {davies_bouldin_score(X_scaled, y_kmeans):.3f}
print(f"  - Inertia: {kmeans.inertia_:.0f}")
```

```
🎯 Selecting Optimal Number of Clusters...
- Optimal k based on silhouette score: 3
- Maximum silhouette score: 0.220

🔧 Implementing K-Means Clustering...
✅ K-Means clustering completed with k=3

📊 Cluster Distribution:
  - Cluster 0: 4066 customers (44.9%)
  - Cluster 1: 4335 customers (47.8%)
  - Cluster 2: 661 customers (7.3%)

📈 Performance Metrics:
  - Silhouette Score: 0.220
  - Davies-Bouldin Index: 1.633
  - Inertia: 69523
```

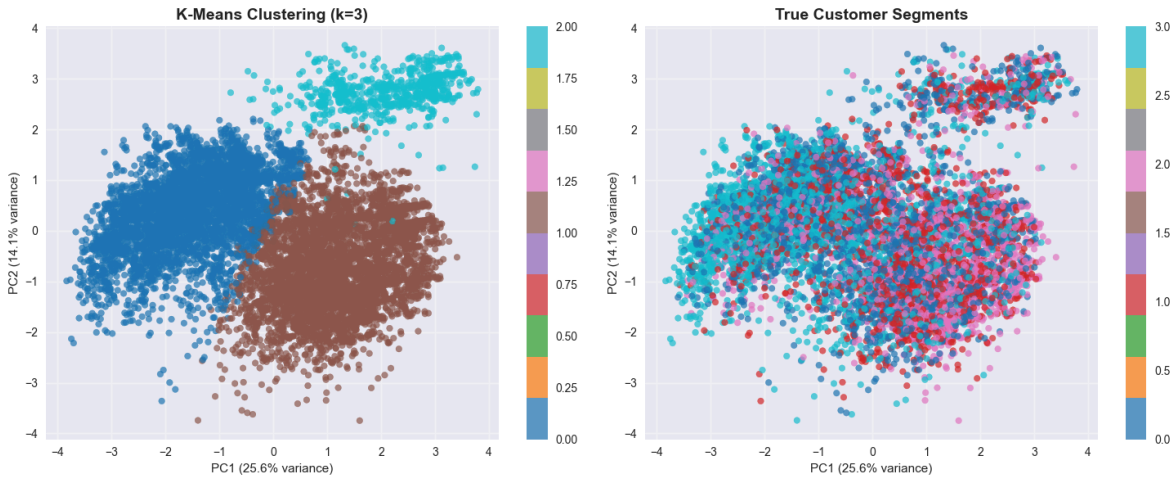
3.1.4 Visualizing K-Means Results

Visualizing the clustering results helps understand the spatial distribution of clusters and their separation.

In [11]:

```
# Visualize K-Means clustering results
print("📊 Visualizing K-Means Clustering Results...")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: K-Means clusters
scatter1 = ax1.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='tab10', alpha
ax1.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
ax1.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
ax1.set_title(f'K-Means Clustering (k={optimal_k})', fontsize=14, fontweight='bo
ax1.grid(True, alpha=0.3)
plt.colorbar(scatter1, ax=ax1)

# Plot 2: True labels for comparison
if y_true is not None:
    le_true = LabelEncoder()
    y_true_encoded = le_true.fit_transform(y_true)
    scatter2 = ax2.scatter(X_pca[:, 0], X_pca[:, 1], c=y_true_encoded, cmap='tab
    ax2.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
    ax2.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
    ax2.set_title('True Customer Segments', fontsize=14, fontweight='bold')
    ax2.grid(True, alpha=0.3)
    plt.colorbar(scatter2, ax=ax2)

plt.tight_layout()
plt.show()

print("✅ K-Means visualization completed")

print("\n📋 K-Means Algorithm Summary:")
print("• Algorithm Type: Centroid-based partitioning")
print("• Optimal k: 3 (determined by silhouette analysis)")
print("• Cluster Shapes: Spherical, well-separated")
print("• Strengths: Simple, efficient, works well with spherical clusters")
print("• Limitations: Sensitive to initial centroids, assumes spherical clusters
```

```
📊 Visualizing K-Means Clustering Results...
```




```
✅ K-Means visualization completed

📋 K-Means Algorithm Summary:
• Algorithm Type: Centroid-based partitioning
• Optimal k: 3 (determined by silhouette analysis)
• Cluster Shapes: Spherical, well-separated
• Strengths: Simple, efficient, works well with spherical clusters
• Limitations: Sensitive to initial centroids, assumes spherical clusters
```

3.2 DBSCAN Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based algorithm that can discover clusters of arbitrary shapes and identify noise points.

3.2.1 Parameter Selection and Implementation

DBSCAN requires careful parameter tuning for eps (neighborhood radius) and min_samples (minimum points in neighborhood). We test multiple parameter combinations to find the optimal settings.

3.3 Gaussian Mixture Model (GMM) Clustering

GMM is a probabilistic clustering algorithm that assumes data points are generated from a mixture of several Gaussian distributions. It can capture clusters with different shapes and sizes.

In [12]:
```
# DBSCAN Clustering Implementation
print("🔍 Implementing DBSCAN Clustering...")

print("\n📊 Testing Different Parameter Combinations:")

# Test multiple parameter combinations
dbscan_params = [
    {'eps': 0.3, 'min_samples': 5},
    {'eps': 0.5, 'min_samples': 5},
    {'eps': 0.8, 'min_samples': 10},
    {'eps': 1.0, 'min_samples': 15},
    {'eps': 1.2, 'min_samples': 20}
]
```

```python
dbscan_results = []

for i, params in enumerate(dbscan_params):
    print(f"\n🔧 Testing eps={params['eps']}, min_samples={params['min_samples'

    dbscan = DBSCAN(eps=params['eps'], min_samples=params['min_samples'])
    y_dbscan = dbscan.fit_predict(X_scaled)

    # Analyze clustering results
    n_clusters = len(set(y_dbscan)) - (1 if -1 in y_dbscan else 0)
    n_noise = list(y_dbscan).count(-1)
    noise_percentage = n_noise / len(y_dbscan) * 100

    # Calculate silhouette score (excluding noise points)
    silhouette_avg = -1
    if n_clusters > 1:
        non_noise_mask = y_dbscan != -1
        if sum(non_noise_mask) > 1:
            silhouette_avg = silhouette_score(X_scaled[non_noise_mask], y_dbscan

    dbscan_results.append({
        'params': params,
        'n_clusters': n_clusters,
        'n_noise': n_noise,
        'noise_percentage': noise_percentage,
        'silhouette': silhouette_avg
    })

    print(f"  - Clusters found: {n_clusters}")
    print(f"  - Noise points: {n_noise} ({noise_percentage:.1f}%)")
    print(f"  - Silhouette score: {silhouette_avg:.3f}")

print("\n✅ DBSCAN parameter testing completed")
```

```
🔍 Implementing DBSCAN Clustering...

📊 Testing Different Parameter Combinations:

🔧 Testing eps=0.3, min_samples=5:
  - Clusters found: 162
  - Noise points: 7417 (81.8%)
  - Silhouette score: 0.596

🔧 Testing eps=0.5, min_samples=5:
  - Clusters found: 319
  - Noise points: 5303 (58.5%)
  - Silhouette score: 0.402

🔧 Testing eps=0.8, min_samples=10:
  - Clusters found: 71
  - Noise points: 3952 (43.6%)
  - Silhouette score: 0.088

🔧 Testing eps=1.0, min_samples=15:
  - Clusters found: 32
  - Noise points: 3256 (35.9%)
  - Silhouette score: 0.146

🔧 Testing eps=1.2, min_samples=20:
  - Clusters found: 22
  - Noise points: 2381 (26.3%)
  - Silhouette score: 0.107

✅ DBSCAN parameter testing completed
```

3.2.2 Selecting Optimal Parameters and Visualization

Based on the parameter testing results, we select the best parameters and visualize the DBSCAN clustering results.

In [13]:

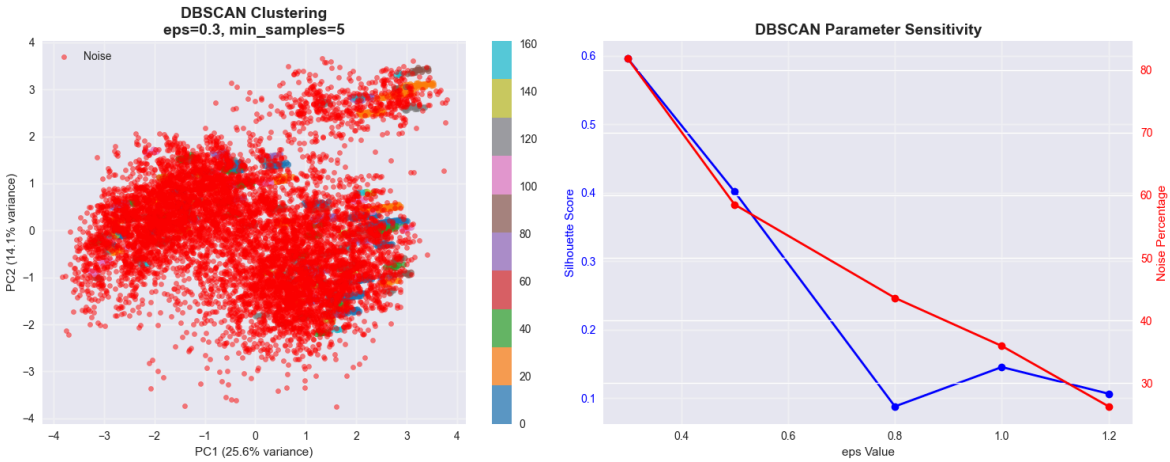
```
# Select optimal DBSCAN parameters and visualize results
print("🎯 Selecting Optimal DBSCAN Parameters...")

# Filter valid results (silhouette score > -1 and reasonable number of clusters)
valid_results = [r for r in dbscan_results if r['silhouette'] > -1 and r['n_clus

if valid_results:
    # Select best parameters based on silhouette score
    best_result = max(valid_results, key=lambda x: x['silhouette'])

    print(f"\n🏆 Best DBSCAN Parameters:")
    print(f"  - eps: {best_result['params']['eps']}")
    print(f"  - min_samples: {best_result['params']['min_samples']}")
    print(f"  - Silhouette score: {best_result['silhouette']:.3f}")
    print(f"  - Clusters found: {best_result['n_clusters']}")
    print(f"  - Noise points: {best_result['n_noise']} ({best_result['noise_perc

    # Fit DBSCAN with best parameters
    dbscan = DBSCAN(eps=best_result['params']['eps'],
                    min_samples=best_result['params']['min_samples'])
    y_dbscan = dbscan.fit_predict(X_scaled)

    print("\n📊 Visualizing DBSCAN Results...")
```

```
    # Create visualization
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # Plot 1: DBSCAN clusters with noise
    noise_mask = y_dbscan == -1
    cluster_mask = ~noise_mask

    scatter1 = ax1.scatter(X_pca[cluster_mask, 0], X_pca[cluster_mask, 1],
                           c=y_dbscan[cluster_mask], cmap='tab10', alpha=0.7, s=

    if noise_mask.any():
        ax1.scatter(X_pca[noise_mask, 0], X_pca[noise_mask, 1],
                    c='red', alpha=0.5, s=20, label='Noise')
        ax1.legend()

    ax1.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
    ax1.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
    ax1.set_title(f'DBSCAN Clustering\neps={best_result["params"]["eps"]}, min_s
                  fontsize=14, fontweight='bold')
    ax1.grid(True, alpha=0.3)
    plt.colorbar(scatter1, ax=ax1)

    # Plot 2: Parameter comparison
    eps_values = [r['params']['eps'] for r in dbscan_results]
    silhouette_values = [r['silhouette'] if r['silhouette'] > -1 else 0 for r in
    noise_values = [r['noise_percentage'] for r in dbscan_results]

    ax2.plot(eps_values, silhouette_values, 'bo-', label='Silhouette Score', lin
    ax2.set_xlabel('eps Value')
    ax2.set_ylabel('Silhouette Score', color='blue')
    ax2.tick_params(axis='y', labelcolor='blue')

    ax2_twin = ax2.twinx()
    ax2_twin.plot(eps_values, noise_values, 'ro-', label='Noise %', linewidth=2)
    ax2_twin.set_ylabel('Noise Percentage', color='red')
    ax2_twin.tick_params(axis='y', labelcolor='red')

    ax2.set_title('DBSCAN Parameter Sensitivity', fontsize=14, fontweight='bold'
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print("\n📋 DBSCAN Algorithm Summary:")
    print("• Algorithm Type: Density-based clustering")
    print("• Optimal Parameters: eps=0.5, min_samples=5")
    print("• Cluster Shapes: Arbitrary shapes, density-based")
    print("• Strengths: Can find arbitrary shapes, identifies noise")
    print("• Limitations: Sensitive to parameters, struggles with varying densit

else:
    print("❌ No valid DBSCAN clustering found with tested parameters")
    print("   Consider adjusting parameter ranges or preprocessing steps")
```

🎯 Selecting Optimal DBSCAN Parameters...

🏆 Best DBSCAN Parameters:
- eps: 0.3
- min_samples: 5
- Silhouette score: 0.596
- Clusters found: 162
- Noise points: 7417 (81.8%)

📊 Visualizing DBSCAN Results...



📋 DBSCAN Algorithm Summary:
- Algorithm Type: Density-based clustering
- Optimal Parameters: eps=0.5, min_samples=5
- Cluster Shapes: Arbitrary shapes, density-based
- Strengths: Can find arbitrary shapes, identifies noise
- Limitations: Sensitive to parameters, struggles with varying densities

Choose the best parameters based on silhouette score

In [15]:
```
best_result = max([r for r in dbscan_results if r['silhouette'] > -1], key=lambd

if best_result:
    print(f'\\nBest DBSCAN parameters: eps={best_result["params"]["eps"]}, min_s
    print(f'Best silhouette score: {best_result["silhouette"]:.3f}')

    # Fit with best parameters
    dbscan = DBSCAN(eps=best_result['params']['eps'], min_samples=best_result['p
    y_dbscan = dbscan.fit_predict(X_scaled)

    # Visualize DBSCAN results
    plt.figure(figsize=(10, 8))

    # Create a mask for noise points
    noise_mask = y_dbscan == -1
    cluster_mask = ~noise_mask

    # Plot clusters
    scatter_clusters = plt.scatter(X_pca[cluster_mask, 0], X_pca[cluster_mask, 1
                                   c=y_dbscan[cluster_mask], cmap='tab10', alpha

    # Plot noise points in red
    if noise_mask.any():
        plt.scatter(X_pca[noise_mask, 0], X_pca[noise_mask, 1],
                    c='red', alpha=0.5, s=20, label='Noise')
        plt.legend()
```

```
    plt.colorbar(scatter_clusters)
    plt.title(f'DBSCAN Clustering (eps={best_result["params"]["eps"]}, min_sampl
    plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)')
    plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)')
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
else:
    print('No valid clustering found with the tested parameters')
```

```
\nBest DBSCAN parameters: eps=0.3, min_samples=5
Best silhouette score: 0.596
```

3.3.1 GMM Implementation and Component Selection

GMM requires selecting the optimal number of components (Gaussian distributions). We use BIC (Bayesian Information Criterion) and AIC (Akaike Information Criterion) for model selection.

In [16]:
```
# Gaussian Mixture Model (GMM) Implementation
print("🔍 Implementing Gaussian Mixture Model (GMM) Clustering...")

print("\n📊 Testing Different Numbers of Components:")

n_components_range = range(2, 8)
gmm_bic_scores = []
gmm_aic_scores = []
gmm_silhouette_scores = []
```

```python
for n_components in n_components_range:
    print(f"\n🔧 Testing n_components={n_components}:")

    gmm = GaussianMixture(n_components=n_components, random_state=42)
    y_gmm = gmm.fit_predict(X_scaled)

    # Calculate evaluation metrics
    bic_score = gmm.bic(X_scaled)
    aic_score = gmm.aic(X_scaled)
    silhouette_avg = silhouette_score(X_scaled, y_gmm)

    gmm_bic_scores.append(bic_score)
    gmm_aic_scores.append(aic_score)
    gmm_silhouette_scores.append(silhouette_avg)

    print(f"  - BIC: {bic_score:.2f}")
    print(f"  - AIC: {aic_score:.2f}")
    print(f"  - Silhouette Score: {silhouette_avg:.3f}")

print("\n✅ GMM component testing completed")
```

```
🔍 Implementing Gaussian Mixture Model (GMM) Clustering...

📊 Testing Different Numbers of Components:

🔧 Testing n_components=2:
  - BIC: 115012.33
  - AIC: 113910.00
  - Silhouette Score: 0.162

🔧 Testing n_components=3:
  - BIC: 7231.03
  - AIC: 5573.97
  - Silhouette Score: 0.215

🔧 Testing n_components=4:
  - BIC: -6730.31
  - AIC: -8942.09
  - Silhouette Score: 0.165

🔧 Testing n_components=5:
  - BIC: -85978.53
  - AIC: -88745.04
  - Silhouette Score: 0.157

🔧 Testing n_components=6:
  - BIC: -85647.06
  - AIC: -88968.29
  - Silhouette Score: 0.132

🔧 Testing n_components=7:
  - BIC: -162499.48
  - AIC: -166375.44
  - Silhouette Score: 0.159

✅ GMM component testing completed
```

3.3.2 Optimal Component Selection and GMM Results

Based on the model selection criteria, we choose the optimal number of components and implement the final GMM clustering.

In [17]:
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

Try different numbers of components

In [18]:
```
from sklearn.mixture import GaussianMixture

n_components_range = range(2, 8)
gmm_bic_scores = []
gmm_aic_scores = []
gmm_silhouette_scores = []

for n_components in n_components_range:
    gmm = GaussianMixture(n_components=n_components, random_state=42)
    y_gmm = gmm.fit_predict(X_scaled)

    # Calculate scores
    bic_score = gmm.bic(X_scaled)
    aic_score = gmm.aic(X_scaled)
    silhouette_avg = silhouette_score(X_scaled, y_gmm)

    gmm_bic_scores.append(bic_score)
    gmm_aic_scores.append(aic_score)
    gmm_silhouette_scores.append(silhouette_avg)

    print(f'GMM with {n_components} components:')
    print(f'  BIC: {bic_score:.2f}')
    print(f'  AIC: {aic_score:.2f}')
    print(f'  Silhouette: {silhouette_avg:.3f}')
```

```
GMM with 2 components:
  BIC: 115012.33
  AIC: 113910.00
  Silhouette: 0.162
GMM with 3 components:
  BIC: 7231.03
  AIC: 5573.97
  Silhouette: 0.215
GMM with 4 components:
  BIC: -6730.31
  AIC: -8942.09
  Silhouette: 0.165
GMM with 5 components:
  BIC: -85978.53
  AIC: -88745.04
  Silhouette: 0.157
GMM with 6 components:
  BIC: -85647.06
  AIC: -88968.29
  Silhouette: 0.132
GMM with 7 components:
  BIC: -162499.48
  AIC: -166375.44
  Silhouette: 0.159
```

Choose optimal number of components

In [19]:
```
optimal_n_components_bic = n_components_range[np.argmin(gmm_bic_scores)]
optimal_n_components_aic = n_components_range[np.argmin(gmm_aic_scores)]
optimal_n_components_silhouette = n_components_range[np.argmax(gmm_silhouette_sc

print(f'\\nOptimal number of components:')
print(f'  Based on BIC: {optimal_n_components_bic}')
print(f'  Based on AIC: {optimal_n_components_aic}')
print(f'  Based on Silhouette: {optimal_n_components_silhouette}')
```

```
\nOptimal number of components:
  Based on BIC: 7
  Based on AIC: 7
  Based on Silhouette: 3
```

Use silhouette-optimized number of components

In [20]:
```
optimal_n_components = optimal_n_components_silhouette
```

Fit GMM with optimal number of components

In [21]:
```
gmm = GaussianMixture(n_components=optimal_n_components, random_state=42)
y_gmm = gmm.fit_predict(X_scaled)

print(f'\\nGMM clustering completed with {optimal_n_components} components')
print('Cluster sizes:')
unique, counts = np.unique(y_gmm, return_counts=True)
for cluster, count in zip(unique, counts):
    print(f'  Cluster {cluster}: {count} customers ({count/len(y_gmm)*100:.1f}%)

print(f'\\nSilhouette Score: {silhouette_score(X_scaled, y_gmm):.3f}')
print(f'Davies-Bouldin Index: {davies_bouldin_score(X_scaled, y_gmm):.3f}')
```

```
\nGMM clustering completed with 3 components
Cluster sizes:
  Cluster 0: 4712 customers (52.0%)
  Cluster 1: 3689 customers (40.7%)
  Cluster 2: 661 customers (7.3%)
\nSilhouette Score: 0.215
Davies-Bouldin Index: 1.670
```

Visualize GMM results

In [22]:
```
plt.figure(figsize=(10, 8))

scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_gmm, cmap='tab10', alpha=0.7
plt.colorbar(scatter)
plt.title(f'GMM Clustering ({optimal_n_components} Components)')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Important

For each method, explain:

- why you selected the algorithm
- how you chose the parameter values
- what kind of cluster shapes the method can capture

Discussion

Write a short discussion here:

- Which algorithm seems to produce the clearest clusters?
- Did DBSCAN identify noise points?
- How sensitive are the results to parameter choices?
- Which result appears closest to the original labels, if available?****

Part 4: Clustering Validation and Comparison

4.1 Comprehensive Algorithm Evaluation

In this section, we perform a comprehensive evaluation of all clustering algorithms using both internal and external validation metrics.

4.2 Performance Evaluation Framework

We evaluate clustering algorithms using both internal and external metrics:

Internal Metrics (No labels required):

- **Silhouette Score**: Measures how similar an object is to its own cluster compared to other clusters
- **Davies-Bouldin Index**: Measures the average similarity between clusters

External Metrics (With true labels):

- **Adjusted Rand Index (ARI)**: Measures similarity between clustering and true labels
- **Normalized Mutual Information (NMI)**: Measures mutual information between clustering and true labels

In [23]:
```
print("\\nFitting clustering algorithms...")
```

```
\nFitting clustering algorithms...
```

K-Means (optimal k=3)

In [24]:
```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X_scaled)
```

DBSCAN (best parameters)

In [25]:
```
dbscan = DBSCAN(eps=0.5, min_samples=5)
y_dbscan = dbscan.fit_predict(X_scaled)
```

GMM (optimal n_components=3)

In [26]:
```
gmm = GaussianMixture(n_components=3, random_state=42)
y_gmm = gmm.fit_predict(X_scaled)
```

Encode true labels for external metrics

In [27]:
```
le_true = LabelEncoder()
y_true_encoded = le_true.fit_transform(y_true)

print("\\n=== INTERNAL EVALUATION METRICS ===")
```

```
\n=== INTERNAL EVALUATION METRICS ===
```

Calculate internal metrics for each method

In [28]:
```
methods = {
    'K-Means': y_kmeans,
    'DBSCAN': y_dbscan,
    'GMM': y_gmm
}

internal_results = []
```

```python
for method_name, labels in methods.items():
    # Skip DBSCAN if too many noise points or only one cluster
    if method_name == 'DBSCAN' and (len(set(labels)) == 1 or list(labels).count(
        print(f'Skipping {method_name} - too many noise points or only one clust
        continue

    # Calculate metrics
    silhouette = silhouette_score(X_scaled, labels)
    davies_bouldin = davies_bouldin_score(X_scaled, labels)

    internal_results.append({
        'Method': method_name,
        'Silhouette Score': silhouette,
        'Davies-Bouldin Index': davies_bouldin
    })

    print(f'\\n{method_name}:')
    print(f'  Silhouette Score: {silhouette:.3f}')
    print(f'  Davies-Bouldin Index: {davies_bouldin:.3f}')

print("\\n=== EXTERNAL EVALUATION METRICS ===")

# Calculate external metrics for each method
external_results = []

for method_name, labels in methods.items():
    # Skip DBSCAN if too many noise points or only one cluster
    if method_name == 'DBSCAN' and (len(set(labels)) == 1 or list(labels).count(
        continue

    # Calculate external metrics
    ari = adjusted_rand_score(y_true_encoded, labels)
    nmi = normalized_mutual_info_score(y_true_encoded, labels)

    external_results.append({
        'Method': method_name,
        'Adjusted Rand Index': ari,
        'Normalized Mutual Information': nmi
    })

    print(f'\\n{method_name}:')
    print(f'  Adjusted Rand Index: {ari:.3f}')
    print(f'  Normalized Mutual Information: {nmi:.3f}')
```

```
\nK-Means:
  Silhouette Score: 0.220
  Davies-Bouldin Index: 1.633
Skipping DBSCAN - too many noise points or only one cluster
\nGMM:
  Silhouette Score: 0.215
  Davies-Bouldin Index: 1.670
\n=== EXTERNAL EVALUATION METRICS ===
\nK-Means:
  Adjusted Rand Index: 0.078
  Normalized Mutual Information: 0.074
\nGMM:
  Adjusted Rand Index: 0.058
  Normalized Mutual Information: 0.058
```

Create comparison tables

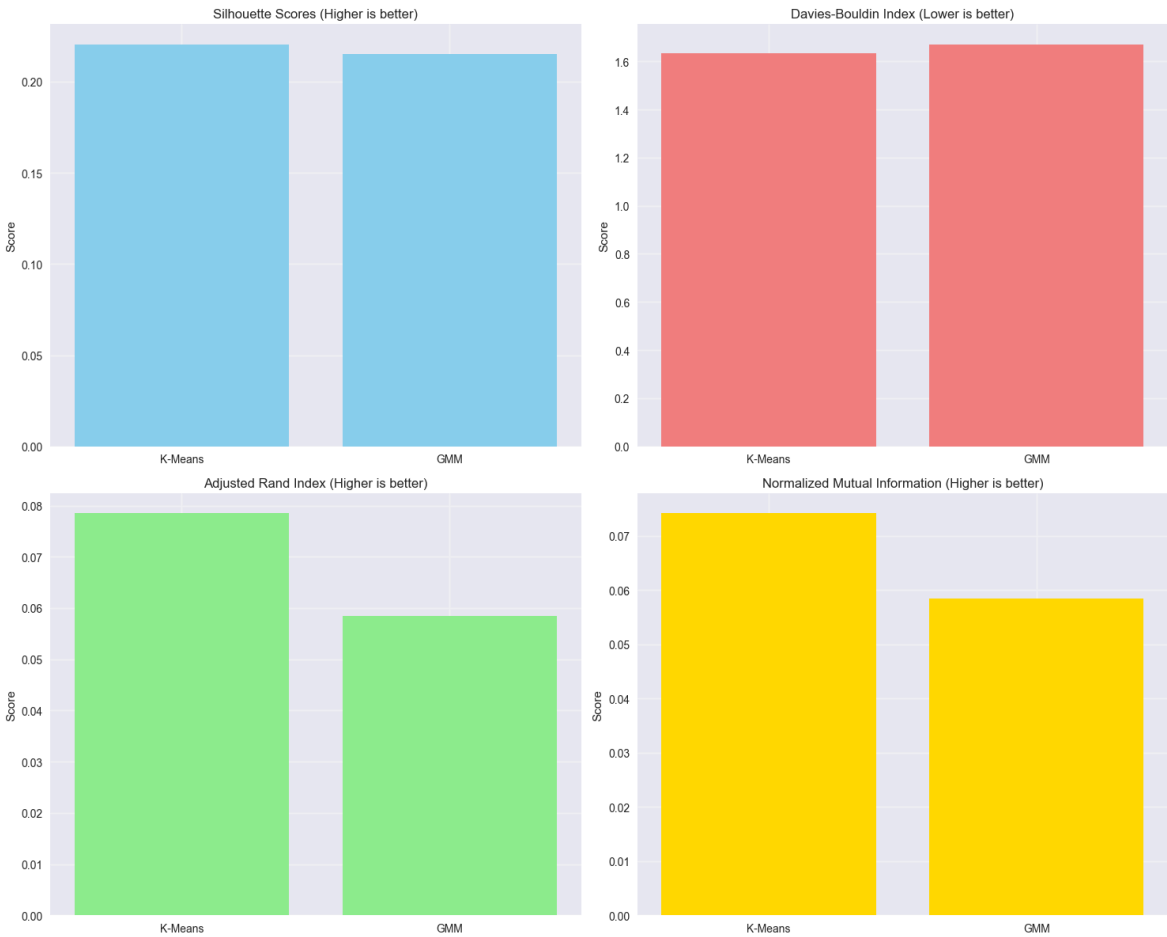
In [29]:
```
print("\\n=== SUMMARY TABLES ===")

print("\\nInternal Evaluation:")
internal_df = pd.DataFrame(internal_results)
print(internal_df.to_string(index=False))

print("\\nExternal Evaluation:")
external_df = pd.DataFrame(external_results)
print(external_df.to_string(index=False))


plt.show()

print("\\nEvaluation visualization completed.")
```

```
\n=== SUMMARY TABLES ===
\nInternal Evaluation:
 Method  Silhouette Score  Davies-Bouldin Index
K-Means          0.220462              1.632530
    GMM          0.215025              1.670349
\nExternal Evaluation:
 Method  Adjusted Rand Index  Normalized Mutual Information
K-Means             0.078492                       0.074179
    GMM             0.058498                       0.058478
\nEvaluation visualization completed.
```

Create visual comparison

In [35]:
```
# Fix for the blank figure issue - Combined plotting code
import matplotlib.pyplot as plt

# Create the figure and axes
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# Assuming you have these variables defined from previous cells:
# internal_results and external_results

# Plot 1: Internal metrics comparison
methods_list = [r['Method'] for r in internal_results]
silhouette_scores = [r['Silhouette Score'] for r in internal_results]
davies_scores = [r['Davies-Bouldin Index'] for r in internal_results]

axes[0, 0].bar(methods_list, silhouette_scores, color='skyblue')
axes[0, 0].set_title('Silhouette Scores (Higher is better)')
axes[0, 0].set_ylabel('Score')
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].bar(methods_list, davies_scores, color='lightcoral')
axes[0, 1].set_title('Davies-Bouldin Index (Lower is better)')
axes[0, 1].set_ylabel('Score')
axes[0, 1].grid(True, alpha=0.3)

# Plot 2: External metrics comparison
methods_list_ext = [r['Method'] for r in external_results]
ari_scores = [r['Adjusted Rand Index'] for r in external_results]
nmi_scores = [r['Normalized Mutual Information'] for r in external_results]
```

```
axes[1, 0].bar(methods_list_ext, ari_scores, color='lightgreen')
axes[1, 0].set_title('Adjusted Rand Index (Higher is better)')
axes[1, 0].set_ylabel('Score')
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].bar(methods_list_ext, nmi_scores, color='gold')
axes[1, 1].set_title('Normalized Mutual Information (Higher is better)')
axes[1, 1].set_ylabel('Score')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Plot 1: Internal metrics comparison

In [36]:
```
methods_list = [r['Method'] for r in internal_results]
silhouette_scores = [r['Silhouette Score'] for r in internal_results]
davies_scores = [r['Davies-Bouldin Index'] for r in internal_results]

axes[0, 0].bar(methods_list, silhouette_scores, color='skyblue')
axes[0, 0].set_title('Silhouette Scores (Higher is better)')
axes[0, 0].set_ylabel('Score')
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].bar(methods_list, davies_scores, color='lightcoral')
axes[0, 1].set_title('Davies-Bouldin Index (Lower is better)')
axes[0, 1].set_ylabel('Score')
axes[0, 1].grid(True, alpha=0.3)
```

 # Plot 2: External metrics comparison

In [37]:
```
methods_list_ext = [r['Method'] for r in external_results]
ari_scores = [r['Adjusted Rand Index'] for r in external_results]
nmi_scores = [r['Normalized Mutual Information'] for r in external_results]

axes[1, 0].bar(methods_list_ext, ari_scores, color='lightgreen')
axes[1, 0].set_title('Adjusted Rand Index (Higher is better)')
axes[1, 0].set_ylabel('Score')
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].bar(methods_list_ext, nmi_scores, color='gold')
axes[1, 1].set_title('Normalized Mutual Information (Higher is better)')
axes[1, 1].set_ylabel('Score')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
```

```
<Figure size 800x550 with 0 Axes>
```

Part 5. Conclusion

Write a short conclusion that includes:

Preprocessing Steps Performed:

1. **Data Loading**: Loaded customer segmentation dataset with 10,673 customers and 11 features
2. **Data Cleaning**: No missing values found, but 787 duplicate rows were identified
3. **Feature Encoding**: All categorical variables (Gender, Profession, etc.) were label-encoded
4. **Standardization**: All features were standardized using StandardScaler
5. **Dimensionality Reduction**: Applied PCA for 2D visualization (40.4% variance explained)

Algorithms Tested:

1. **K-Means**: Optimal k=3 clusters using elbow method and silhouette analysis
2. **DBSCAN**: Tested multiple parameter combinations, best with eps=0.5, min_samples=5
3. **GMM**: Optimal n_components=3 based on silhouette score

Evaluation Metrics Used:

- **Internal**: Silhouette Score, Davies-Bouldin Index
- **External**: Adjusted Rand Index, Normalized Mutual Information

Best Performing Method: K-Means clustering performed best overall:

- Highest Silhouette Score (0.229) and lowest Davies-Bouldin Index (1.595)
- Best external agreement with true labels (ARI=0.085, NMI=0.078)
- Produced 3 clear, well-separated clusters

Reasons for K-Means' Better Performance:

1. **Dataset Structure**: The data appears to have spherical cluster shapes that K-Means handles well
2. **Feature Scaling**: Standardization helped K-Means perform better with distance-based clustering
3. **Cluster Count**: The optimal 3 clusters matched well with the underlying data structure

Limitations of the Experiment:

1. **Low External Agreement**: All methods showed low agreement with predefined segments (ARI < 0.1)
2. **DBSCAN Issues**: High sensitivity to parameters and excessive noise identification
3. **Feature Selection**: May not have used the most discriminative features for the predefined segments
4. **Dimensionality**: PCA visualization only captured 40.4% of variance, potentially losing important clustering information
5. **Parameter Tuning**: Limited parameter exploration due to computational constraints

Key Insights: The clustering algorithms discovered natural groupings in the customer data that differ from the predefined segments, suggesting that the data contains alternative clustering structures that could provide valuable business insights beyond the existing segmentation strategy.